Architecture offline-first et deux stacks simples

Logiciel de caisse pour restaurant

Note de décision — version 1 — 2 juin 2026

Recommandation principale
Retenir un **client épais local** sur un **POS ou une tablette Windows** et comparer deux solutions. Pour livrer une première version exploitable avec le moins de complexité, privilégier **Python + PySide6 + SQLite**. Conserver **Electron + React + TypeScript + SQLite** comme alternative si l'apprentissage d'une application desktop basée sur les technologies web est prioritaire. Dans les deux cas, une vente reste possible sans Internet.

1. Objectif et principe

Le logiciel doit rester simple à développer, installer et maintenir. La première version suit un principe **offline-first** : elle fonctionne sans connexion Internet sur un seul appareil Windows. L'objectif du stage est aussi de progresser en logique métier, en SQL et, selon la stack retenue, en Python ou en TypeScript.

— Une vente est enregistrée dans SQLite sur la caisse avant toute synchronisation éventuelle.

— Le logiciel doit continuer à fonctionner si Internet est indisponible.

— Une sauvegarde manuelle ou planifiée doit pouvoir être exportée sur un autre support.

— Le choix du matériel sera fixé après un test réel d'impression.

Pourquoi cibler Windows dans la première version ?
Python s'exécute et se transforme simplement en application installable sur Windows. Sur Android ou iPadOS, intégrer Python dans une application locale ajoute des difficultés qui n'apportent pas de valeur au premier prototype. Mieux vaut réussir un produit simple sur un OS avant d'envisager un portage.

2. Deux stacks techniques minimalistes

Les deux propositions ciblent Windows et partagent la même base : **SQLite** comme source de vérité, **JSON** pour les tickets figés, **CSV** pour les exports et **Git** pour le suivi du code. Les montants sont stockés en centimes sous forme d'entiers, par exemple `1850` pour 18,50 euros.

Option A — application desktop Python : choix recommandé

Besoin	Choix	Rôle
Logique métier	**Python**	Écrire les règles essentielles : panier, paiements, historique et statistiques simples.
Interface tactile	**PySide6**	Construire une application desktop tactile avec Qt for Python.
Stockage local	**SQLite**	Enregistrer les données dans un fichier local, sans serveur à installer.
Tickets et sauvegardes	**JSON, CSV**	Figer les tickets et produire des exports lisibles.
Exécutable Windows	**pyside6-deploy**	Générer un exécutable Windows à partir de l'application Python.
Tests	**pytest**	Vérifier les calculs critiques et les annulations.
Impression	SDK du fabricant	Piloter l'imprimante après un test sur le matériel réel.

Atout principal : une seule base de code Python, sans API locale et sans build frontend. C'est le chemin le plus court pour obtenir une application installable et se concentrer sur la logique métier.

Option B — application desktop Electron et React

Besoin	Choix	Rôle
Runtime desktop	**Electron**	Exécuter une application desktop locale qui embarque Chromium et Node.js.
Logique métier	**TypeScript**	Écrire les règles essentielles dans un langage typé.
Interface tactile	**React, TypeScript, CSS**	Construire des écrans tactiles structurés en composants réutilisables.
Build interface	**Vite**	Transformer l'interface React en fichiers statiques intégrés à l'application.
Données et matériel	Processus principal Electron via **IPC**	Relier l'interface à SQLite et aux fonctions natives du poste.
Stockage local	**SQLite**	Enregistrer les données dans un fichier local, sans serveur distant.
Tickets et sauvegardes	**JSON, CSV**	Figer les tickets et produire des exports lisibles.
Exécutable Windows	**Electron Forge**	Générer un exécutable ou un installateur Windows.
Tests	**Vitest**	Vérifier les calculs critiques et les annulations.

Atout principal : progresser en React et TypeScript dans une vraie application desktop. Electron permet de conserver une interface web moderne tout en accédant aux fonctions locales par IPC. Cette option reste raisonnable, mais comporte davantage de pièces à assembler.

Comparaison rapide

Critère	Option A — PySide6	Option B — Electron + React
Simplicité de développement	**Très bonne**	Bonne
Nombre de briques principales	Python, PySide6, SQLite	Electron, React, TypeScript, Vite, SQLite
Compétences travaillées	Python, SQL, logique métier	TypeScript, SQL, React, IPC, application desktop
Priorité	**Livrer efficacement**	Approfondir le desktop avec technologies web

3. Principe offline-first et évolution possible

Le site public de Splash présente une caisse connectée, un mode hors ligne fiable, des statistiques, un back-office, des bornes et un écran cuisine. Le détail de leur code n'est pas public. Le schéma ci-dessous est donc une **trajectoire prudente pour ce projet**, pas une description de leur implémentation interne.

Sujet	Première version à livrer	Extension facultative ultérieure
Encaissement	Application desktop et SQLite locale	Identique : le cloud ne bloque jamais une vente
Données	Catalogue, commandes et paiements locaux	File de synchronisation envoyée quand Internet revient
Pilotage	Historique et statistiques sur la caisse	API centrale, base PostgreSQL et back-office web
Périphériques	Imprimante et TPE testés sur le poste	Borne ou écran cuisine ajoutés selon le besoin

Place éventuelle de Next.js
Next.js peut devenir utile plus tard pour un back-office web consultable à distance. Il ne doit pas être l'unique runtime de la caisse au moment de l'encaissement.

4. Compatibilité selon le système

Les deux stacks proposées ciblent Windows. Les autres systèmes restent intéressants à comparer pour le matériel, mais demanderaient un autre choix technique si un portage devenait nécessaire.

Système	Choix technique	Langages principaux	Remarque
Android / SUNMI OS	Non retenu pour la V1	Kotlin ou JavaScript	Bon matériel POS, mais Python local n'est pas le chemin le plus simple.
Windows 11	**Choix recommandé**	Python ou TypeScript	Simple à développer, empaqueter et tester ; compatible avec de nombreux périphériques USB.
iPadOS	Non retenu pour la V1	Swift ou JavaScript	Bon matériel, mais distribution et impression plus contraignantes pour cette première version.

5. Choix conseillé pour commencer

Premier environnement à tester : un appareil Windows 11 disponible
Avant tout achat, construire le prototype desktop sur un ordinateur ou une tablette Windows déjà disponible. Tester ensuite le logiciel sur un terminal POS Windows, par exemple un SUNMI T3 PRO SUPER configuré sous Windows si cette version est disponible auprès du revendeur.

Pourquoi procéder ainsi : le choix définitif du terminal dépend surtout de l'impression, du tiroir-caisse, du prix et de la disponibilité. Le prototype peut avancer sans attendre cet achat.

6. Plan d'action détaillé

L'objectif est de valider chaque bloc avant de passer au suivant. Ce plan donne un ordre de travail, pas un calendrier figé.

1. **Valider les décisions de départ.**
   Choisir l'option A ou B avec le maître de stage, préciser les moyens de paiement, les taux de TVA, le besoin d'impression et le matériel déjà disponible. **Résultat :** une décision technique courte et les questions client prioritaires.

2. **Créer le socle de l'application desktop.**
   Initialiser le dépôt Git, l'arborescence, la configuration et un premier écran tactile lançable. **Résultat :** une application vide mais installable en développement.
3. **Modéliser les données locales.**
   Créer SQLite avec articles, catégories, commandes, lignes de commande, paiements, utilisateurs et identifiants stables. Stocker les montants en centimes. **Résultat :** un schéma versionné et quelques données de démonstration.
4. **Développer catalogue, panier et commandes.**
   Ajouter les articles au panier, modifier les quantités et distinguer sur place, à emporter et livraison. **Résultat :** une commande complète peut être préparée avant encaissement.
5. **Implémenter les encaissements.**
   Gérer espèces, carte, titres-restaurant, autres moyens, paiement mixte et rendu de monnaie. **Résultat :** les calculs critiques sont couverts par des tests automatisés.
6. **Ajouter la traçabilité.**
   Figer un instantané JSON à l'encaissement, conserver l'opérateur et l'horodatage, puis traiter une correction ou une annulation comme une nouvelle opération liée. **Résultat :** aucune vente validée n'est modifiée silencieusement.
7. **Construire l'historique et les statistiques simples.**
   Rechercher les ventes et afficher chiffre d'affaires, volume de transactions, répartition des paiements et totaux journaliers. **Résultat :** le responsable dispose d'un premier tableau de bord local.
8. **Tester l'impression et les périphériques.**
   Imprimer une note sur le matériel réel, gérer l'échec d'impression et valider le besoin de tiroir-caisse. **Résultat :** un compte rendu matériel avant achat définitif.
9. **Ajouter sauvegarde, restauration et livraison Windows.**
   Exporter JSON et CSV, tester la restauration, puis générer l'exécutable ou l'installateur. **Résultat :** une version démontrable hors ligne sur un poste Windows distinct.
10. **Réaliser la recette et la mise en service.**
    Jouer les parcours essentiels avec le client, corriger les défauts bloquants, rédiger un guide court et former les utilisateurs. **Résultat :** une première version exploitable validée.

7. Décisions à prendre avant achat

— La caisse reste-t-elle fixée au comptoir ou doit-elle être déplacée en salle ?

— Une imprimante ticket 80 mm intégrée est-elle souhaitée ?

— Faut-il ouvrir automatiquement un tiroir-caisse ?

— Une imprimante cuisine est-elle nécessaire dès la première version ?

— Quel budget matériel est prévu ?

— Le client possède-t-il déjà une tablette ou un terminal réutilisable ?

8. Comparaison du matériel et ordre de prix

Les modèles ci-dessous sont des pistes de comparaison, pas une commande d'achat. Les prix sont des **ordres de grandeur TTC relevés le 2 juin 2026**, hors accessoires et hors promotions temporaires lorsqu'elles sont identifiables. Ils doivent être revérifiés avant achat. Pour un terminal professionnel sans tarif public stable en France, il faut demander un devis au revendeur.

Android et terminaux POS Android

Modèle	Prix indicatif	Intérêt	Vigilance
Samsung Galaxy Tab A9+	Environ 250 à 350 EUR	Tablette 11 pouces économique avec imprimante externe.	Prévoir support, alimentation continue et imprimante réseau.
Samsung Galaxy Tab Active5 Pro Enterprise Edition	Environ 600 à 850 EUR	Tablette renforcée 10,1 pouces ; protection IP68 et batterie amovible.	Plus coûteuse ; imprimante externe nécessaire.
SUNMI D3 MINI	Environ 400 à 500 EUR	POS Android compact 10,1 pouces avec imprimante thermique 80 mm intégrée.	Ne correspond pas aux stacks Windows retenues pour la V1.

Windows

Modèle	Prix indicatif	Intérêt	Vigilance
Microsoft Surface Pro 12 ou 13 pouces	Environ 1 000 à 1 600 EUR	Tablette Windows polyvalente pour le développement et les essais.	Ajouter support et imprimante ; vérifier les pilotes périphériques sur processeur ARM.
Lenovo ThinkPad X12 Detachable Gen 2	Environ 1 600 à 2 700 EUR	Tablette Windows professionnelle 12 pouces avec clavier détachable.	Robuste mais chère pour une caisse seule ; comparer avec un terminal POS dédié.
SUNMI T3 PRO SUPER	Sur devis	POS fixe dual-OS Windows ou Android, adapté à un comptoir.	Valider la configuration Windows disponible, l'imprimante et les accessoires.

iPadOS

Modèle	Prix indicatif	Intérêt	Vigilance
iPad 11 pouces (A16)	Environ 400 à 650 EUR	Option iPad la plus raisonnable pour une caisse simple.	Prévoir support, alimentation et validation de l’impression.
iPad Air 11 pouces (M4)	Environ 650 à 1 100 EUR	Plus confortable si le client veut un appareil polyvalent.	Plus puissant et plus cher que nécessaire pour la caisse seule.
iPad mini (A17 Pro)	Environ 550 à 900 EUR	Intéressant pour la mobilité ou la prise de commande en salle.	Écran 8,3 pouces trop petit pour un comptoir fixe.

9. Sources consultées

- Python `sqlite3` : documentation
- PySide6 : documentation
- `pyside6-deploy` : documentation
- Splash : site officiel
- Electron : documentation
- Electron : processus principal et rendu : documentation
- Electron Forge : documentation
- React : documentation
- Vite : documentation
- Vitest : documentation
- Next.js : documentation
- SQLite : présentation officielle
- Samsung Galaxy Tab A9+ : Samsung France
- Prix Galaxy Tab A9+ : Fnac
- Galaxy Tab Active5 Pro : Samsung Professionnels France
- Prix Galaxy Tab Active5 Pro : Galaxus France
- SUNMI D3 MINI : fiche produit PDF
- Prix SUNMI D3 MINI : Shop NFC
- SUNMI T3 PRO : SUNMI
- Microsoft Surface Pro : Microsoft France
- Prix Surface Pro : Microsoft Store France
- Lenovo ThinkPad X12 Detachable Gen 2 : Lenovo France
- Prix ThinkPad X12 Gen 2 : PcComponentes France
- Apple iPad 11 pouces (A16) : Apple
- Apple iPad Air : Apple
- Apple iPad mini (A17 Pro) : Apple
- Prix des iPad : Apple Store France

Document de travail. Les modèles cités sont des candidats à tester. La décision d’achat dépend de la disponibilité en France, du prix obtenu auprès du revendeur et de la validation de l’impression sur le matériel réel.